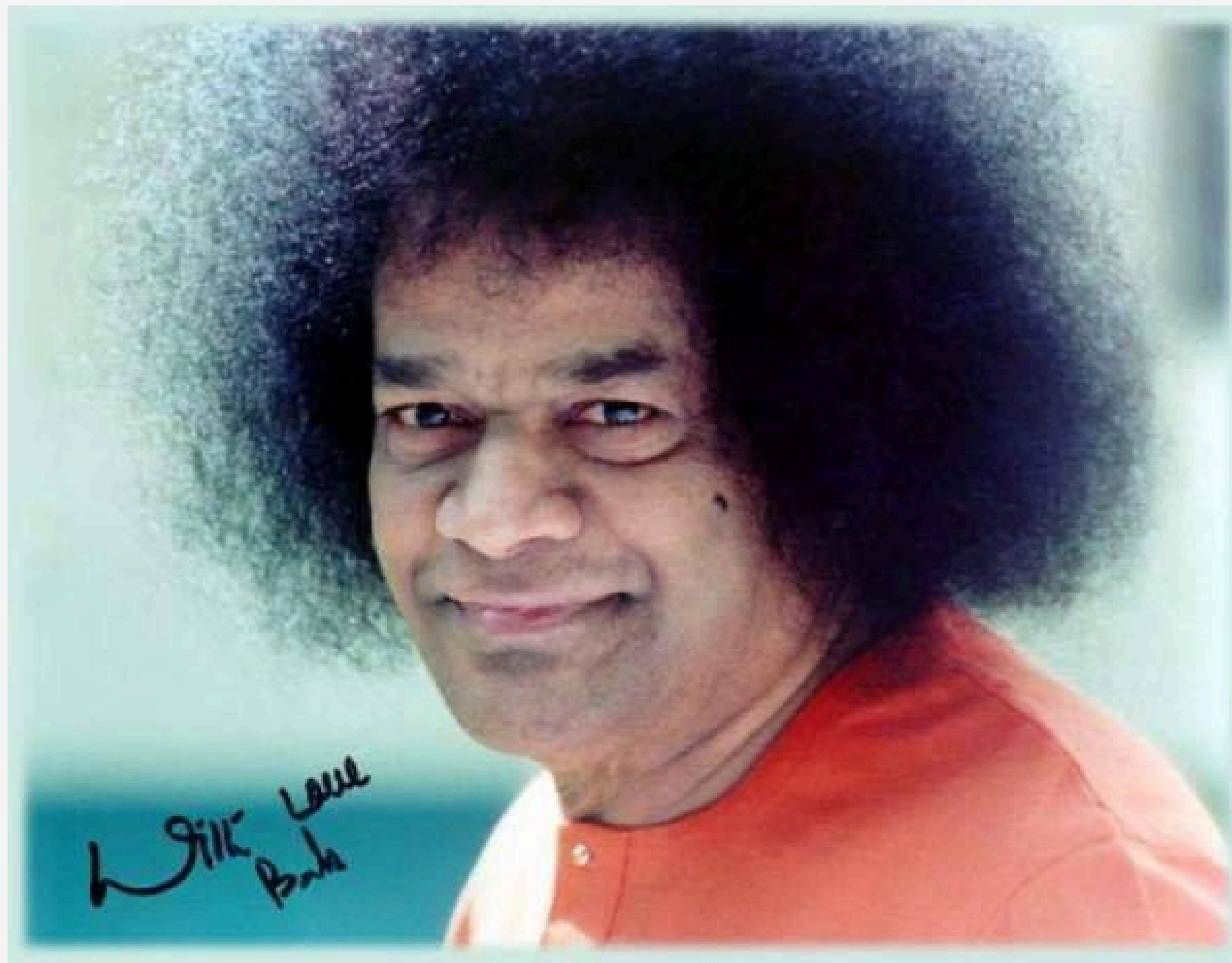

MINI PROJECT PRESENTATION

PROCESS SCHEDULING AND INSIGHTS WITH BPFTRACE

Yedu Krishna P
I MSc Mathematics
Regno: 23018

Guided by:
Shaju Abraham,
Naveen Miriyalu,
Dibyam Pradhan,
Dr. Raghunatha Sarma



OFFERING THE EFFORTS AT HIS FEET

CHAPTER I - INTRODUCTION

SCHEDULERS

In OS, the Scheduler refers to the program that is responsible for the **removal of the currently executing process, selection of the next process** and the **context switch** that is involved in changing the states of these processes.

Goals of a Scheduler:

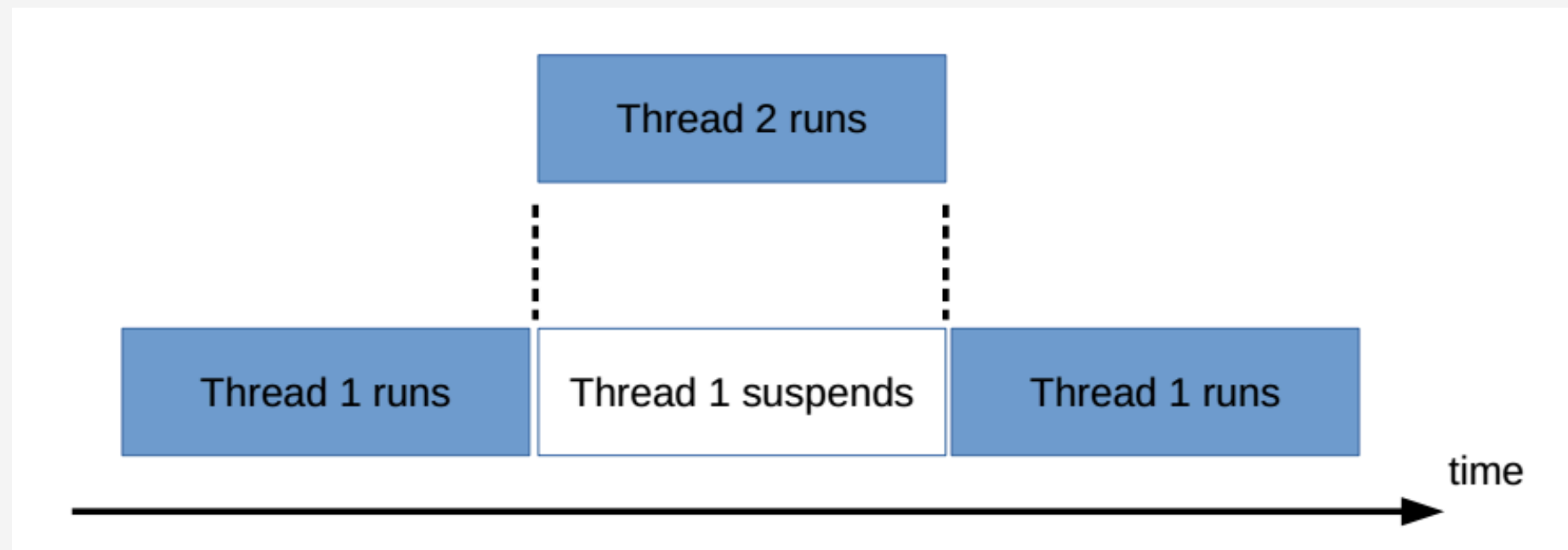
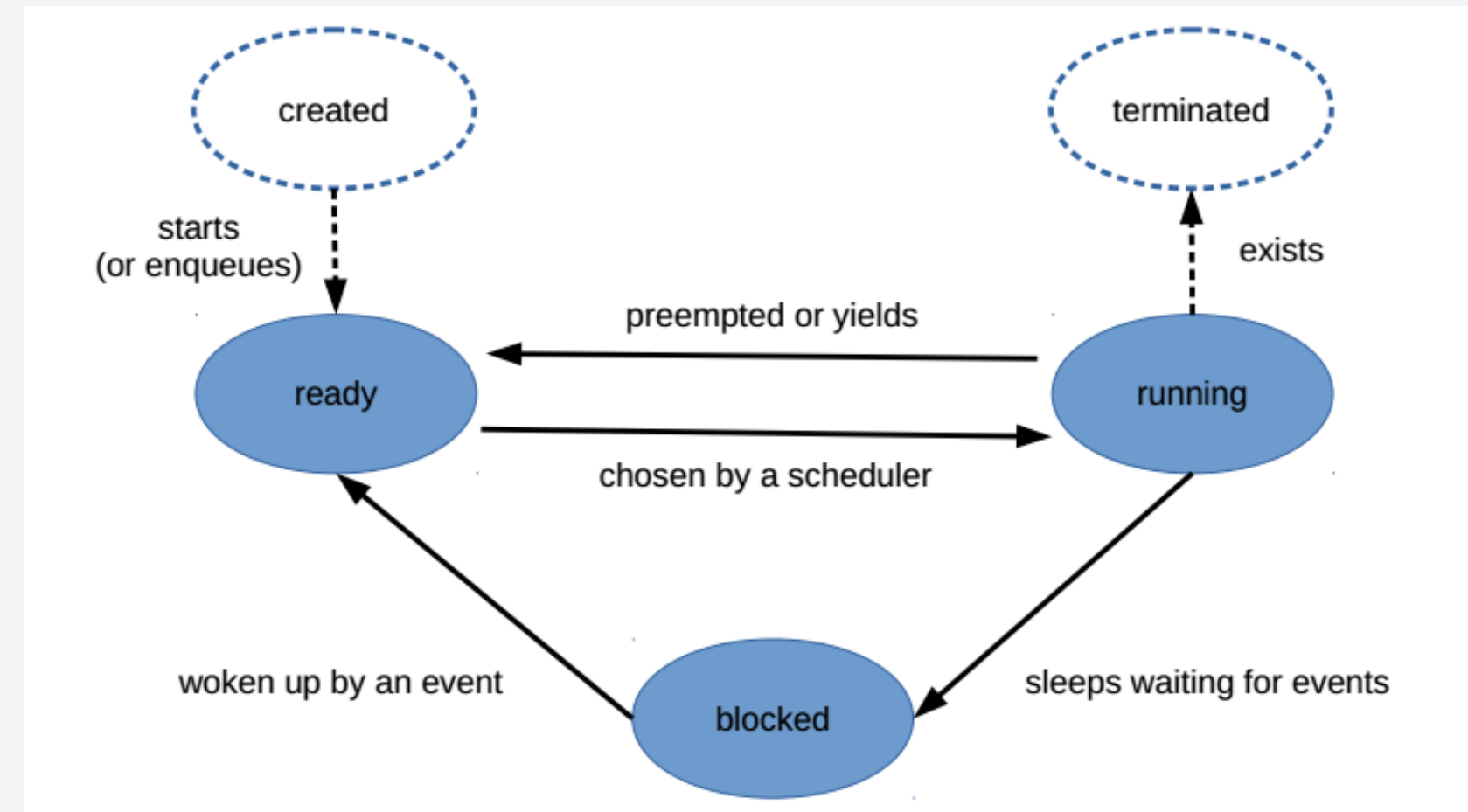
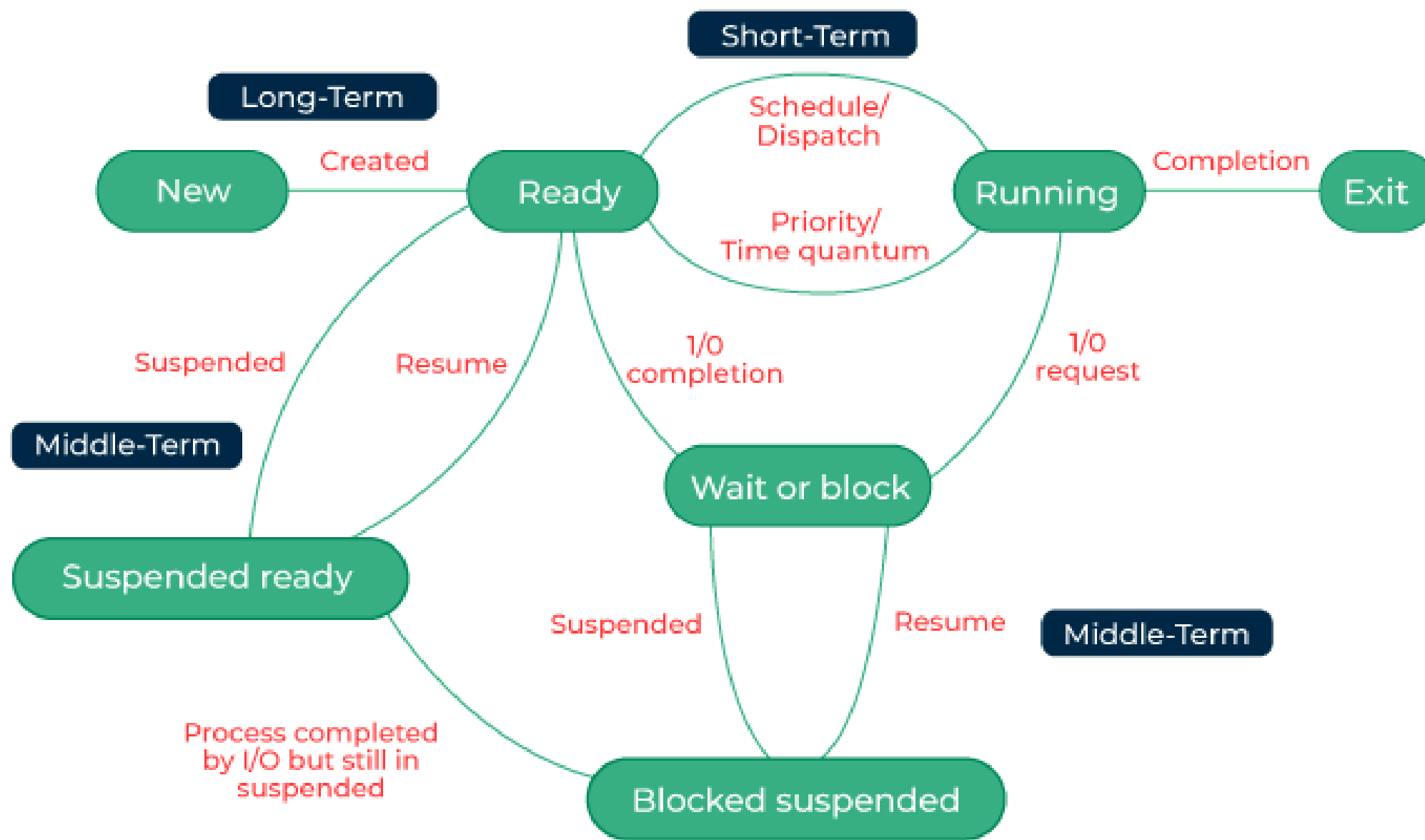
- Fast process response time
 - Good throughput for background processes
 - Avoidance of process starvation
 - Reconciliation of the needs of low and high-priority processes
-

SCHEDULERS

Scheduling policy - Set of rules that determine the 'when' and 'how' to select processes to run. Eg: FIFO, SJF, Round Robin

Depending on the OS and the scheduling policies, the processes may be:

- **Preemptable** - Allows for switching process in middle of execution (timer can interrupt => Time Sharing).
- **Non Preemptable** - Relinquishes CPU only voluntarily.



PROCESSES

Process - Any running instance of a program.

Thread - Basic unit of execution that has a single flow of execution.

Task - Set of program instructions loaded onto the memory.

Classes of Processes:

- **Interactive** - Interact constantly with their users, waits a lot for input.
- **Batch** (Background) - eg: Bookkeeping processes
- **Real Time** - High priority tasks that require immediate response from the CPU (eg: Sensor data)

Types of Processes - CPU Bound or IO Bound.

CHAPTER II - LINUX AND SCHEDULERS

PROCESSES IN LINUX

Modern OSes segregate the virtual memory into user space and kernel space to provide memory protection.

- In Linux, processes are represented and stored using the **task_struct** structure (Process Descriptor).
- Each process is given a User mode and Kernel mode process stack.
 - **User stack** - used by the process itself to execute the program
 - **Kernel stack** - used by kernel when system calls are made etc.
- The Scheduler in Linux is Preemptable and hence we need Synchronization mechanisms like mutex locks etc.

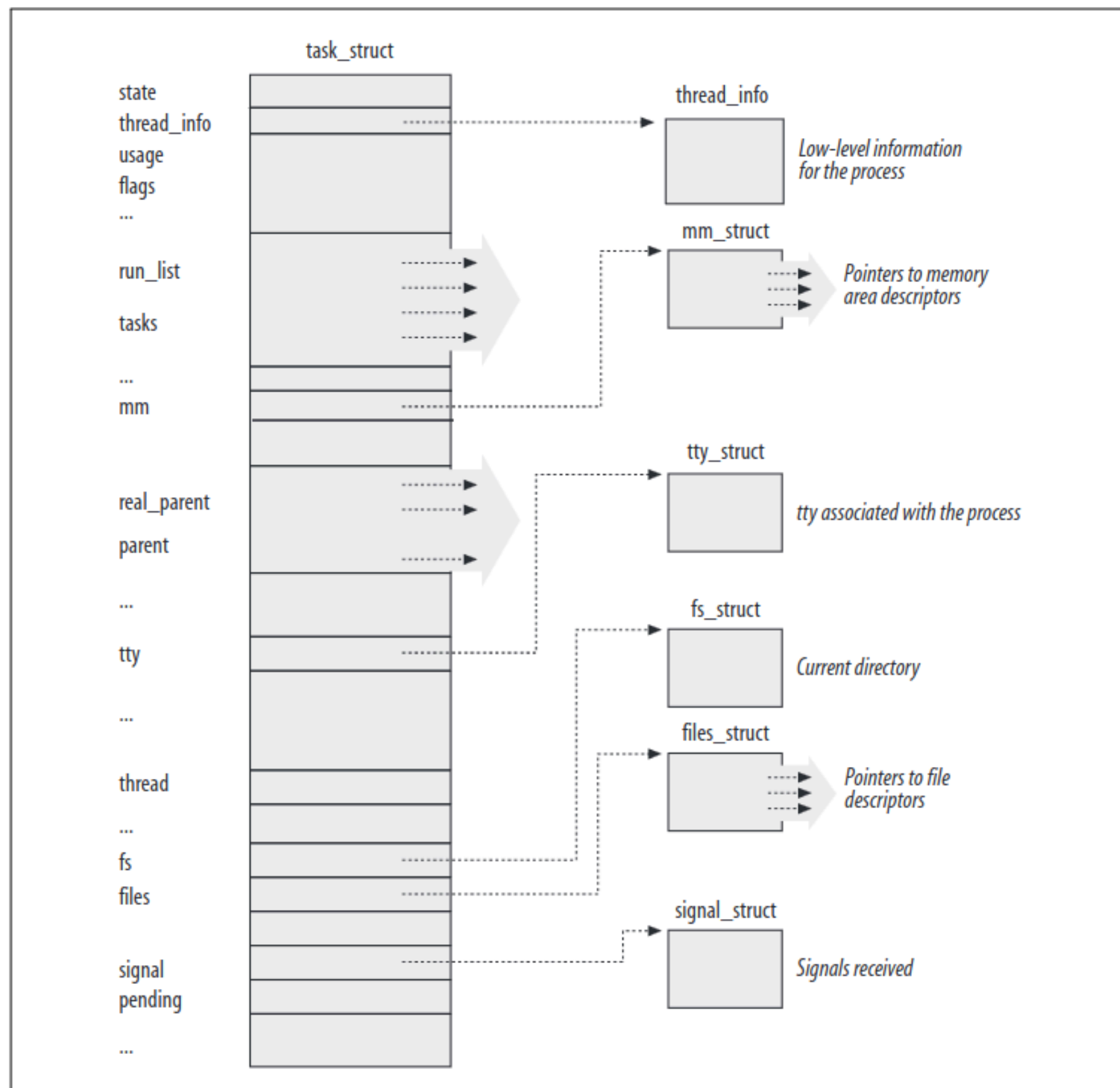


Figure 3-1. The Linux process descriptor

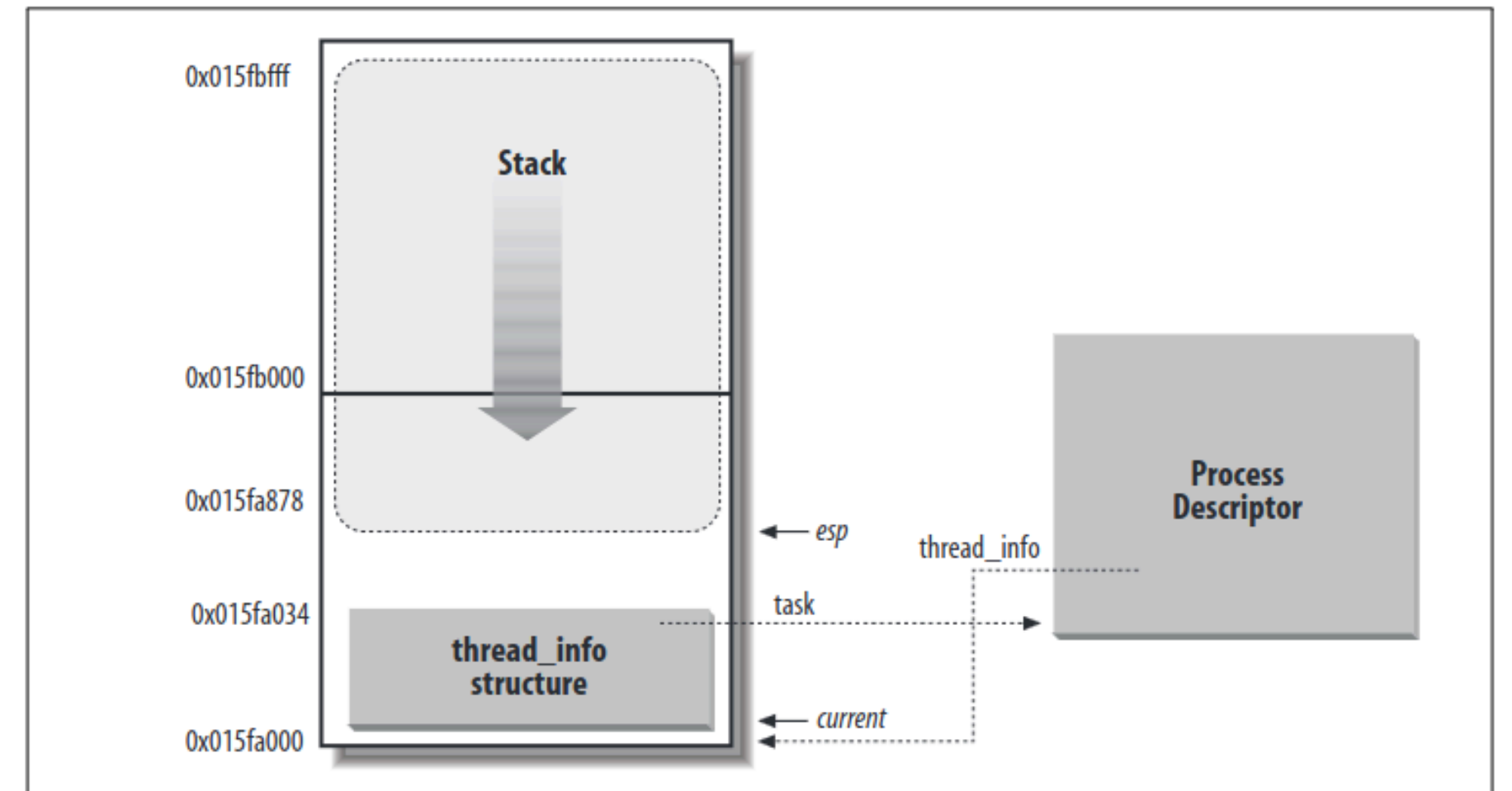
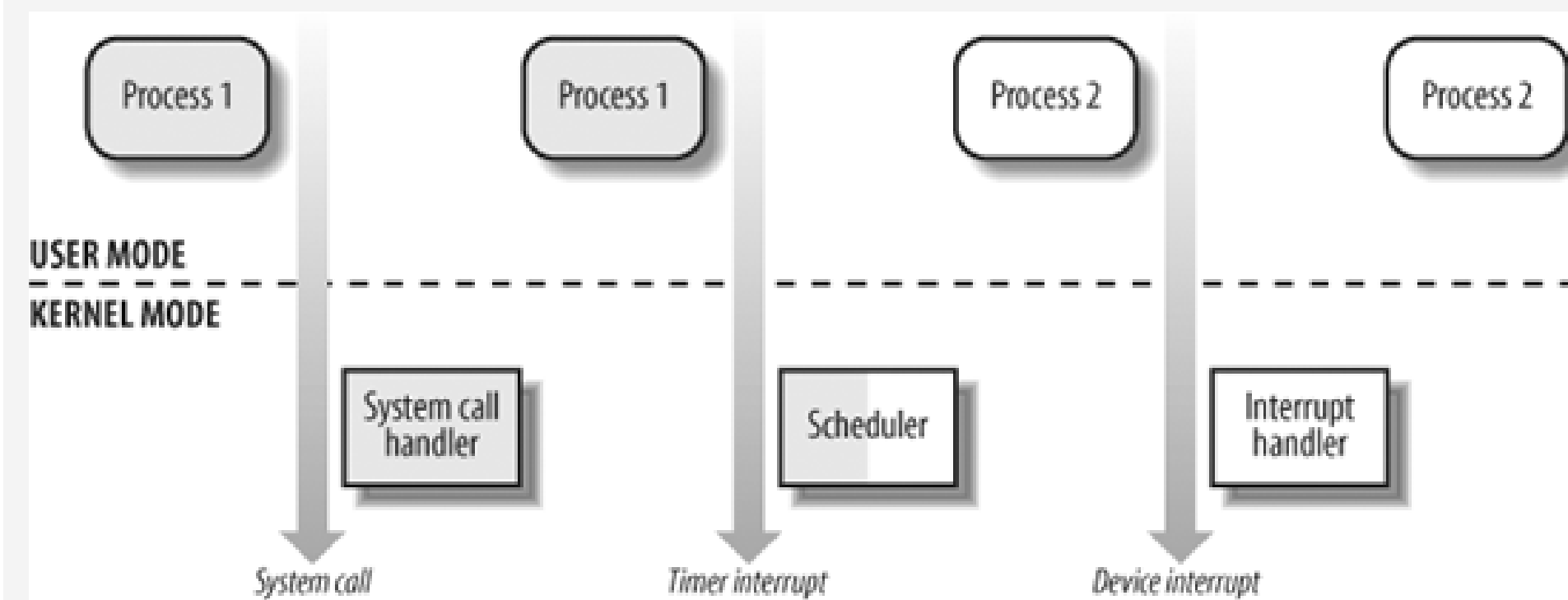


Figure 3-2. Storing the `thread_info` structure and the process kernel stack in two page frames



SCHEDULING IN LINUX

The scheduling policy used for a process depends on its class.

The Linux Kernel uses:

SCHED_NORMAL for Interactive and Batch Processes. (Time Sharing using CFS)

SCHED_FIFO or **SCHED_RR** for Real Time Processes.

SCHED_FIFO – Process is preempted only when higher priority task arrives. Can use CPU as long as it wants even when there are other RT processes ready.

SCHED_RR – Fair Round-Robin assignment of CPU for all the processes with same priority. (Lower priority tasks are executed only when higher priority tasks are done)

When a process is ready to execute, it enters the TASK_RUNNING state. The processes are chosen for execution based on their priorities.

Static Priority:

- Value between 100 (highest) – 139 (least).
- When a process is created, it inherits it's parent's static priority.
Can be set with nice() and setpriority().
- Used to determine the Base Time Quantum that is assigned when a process exhausts the previous quantum.

$$\begin{array}{l} \text{base time quantum} \\ \text{(in milliseconds)} \end{array} = \begin{cases} (140 - \text{static priority}) \times 20 & \text{if static priority} < 120 \\ (140 - \text{static priority}) \times 5 & \text{if static priority} \geq 120 \end{cases}$$

Dynamic Priority:

- Dynamic Priority = $\max(100, \min(\text{static priority} - \text{bonus} + 5, 139))$
- **This determines the process that is to be selected next to run.**
- A process can be considered as interactive if it satisfies the formula:

$$\text{dynamic priority} \leq 3 \times \text{static priority} / 4 + 28$$

or equivalently

$$\text{bonus} - 5 \geq \text{static priority} / 4 - 28$$

To ensure even low priority processes are executed, Linux marks the process as either Active process (yet to complete base quantum) or Expired process (completed its quantum).

Description	Static priority	Nice value	Base time quantum	Interactivedelta	Sleep time threshold
Highest static priority	100	-20	800 ms	-3	299 ms
High static priority	110	-10	600 ms	-1	499 ms
Default static priority	120	0	100 ms	+2	799 ms
Low static priority	130	+10	50 ms	+4	999 ms
Lowest static priority	139	+19	5 ms	+6	1199 ms

Average sleep time	Bonus
Greater than or equal to 0 but smaller than 100 ms	0
Greater than or equal to 100 ms but smaller than 200 ms	1
Greater than or equal to 200 ms but smaller than 300 ms	2
Greater than or equal to 300 ms but smaller than 400 ms	3
Greater than or equal to 400 ms but smaller than 500 ms	4
Greater than or equal to 500 ms but smaller than 600 ms	5
Greater than or equal to 600 ms but smaller than 700 ms	6
Greater than or equal to 700 ms but smaller than 800 ms	7
Greater than or equal to 800 ms but smaller than 900 ms	8
Greater than or equal to 900 ms but smaller than 1000 ms	9
1 second	10

Real Time Processes:

- RT Processes have priorities from 1 – 99. Always set to Active.
- RT processes are replaced only when
 - It voluntarily relinquishes the CPU
 - It is preempted by a process with higher priority
 - Blocks on IO and is put to sleep
 - Task is stopped or killed
 - It's quantum is exhausted if policy is SCHED_RR

When a process needs to be rescheduled or it's time quantum expires, the TIF_NEED_RESCHED flag is set in the thread_info struct and scheduler is invoked.

SCHEDULING IN LINUX

The Scheduler in Linux uses the following data structures and functions in addition to the Process desc:

- Struct sched_class
- Struct rq (for runqueue of a CPU)
- Function __schedule()
- Function scheduler_tick()

Overview of Process Descriptors

Type	Name	Description
unsigned long	state	The current state of the process
int	prio	Dynamic priority of the process
int	static_prio	Static priority of the process
struct list_head	run_list	Pointers to the next and previous elements in the run-queue list to which the process belongs
prio_array_t *	array	Pointer to the runqueue's prio_array_t set that includes the process
unsigned long	sleep_avg	Average sleep time of the process
unsigned long long	timestamp	Time of last insertion of the process in the runqueue, or time of last process switch involving the process
unsigned long long	last_ran	Time of last process switch that replaced the process
int	activated	Condition code used when the process is awakened
unsigned long	policy	The scheduling class of the process (SCHED_NORMAL, SCHED_RR, or SCHED_FIFO)
cpumask_t	cpus_allowed	Bit mask of the CPUs that can execute the process
unsigned int	time_slice	Ticks left in the time quantum of the process
unsigned int	first_time_slice	Flag set to 1 if the process never exhausted its time quantum
unsigned long	rt_priority	Real-time priority of the process

Overview of sched_class

```
struct sched_class {  
    const struct sched_class *next;  
    ...  
    struct task_struct * (*pick_next_task) (struct rq *rq, struct task_struct *prev);  
    void (*put_prev_task) (struct rq *rq, struct task_struct *p);  
    ...  
    void (*task_tick) (struct rq *rq, struct task_struct *p, int queued);  
    ...  
};
```

```
const struct sched_class rt_sched_class = {  
    .next          = &fair_sched_class,  
    ...  
    .pick_next_task    = pick_next_task_rt,  
    .put_prev_task     = put_prev_task_rt,  
    ...  
    .task_tick        = task_tick_rt,  
    ...  
};
```

```
const struct sched_class fair_sched_class = {  
    .next          = &idle_sched_class,  
    ...  
    .pick_next_task    = pick_next_task_fair,  
    .put_prev_task     = put_prev_task_fair,  
    ...  
    .task_tick        = task_tick_fair,  
    ...  
};
```

Overview of rq

Every CPU (core) has its own runqueue and every process can belong to only one runqueue at a time.

```
struct rq {  
    ...  
    struct cfs_rq cfs;  
    struct rt_rq rt;  
    ...  
    struct task_struct *curr, *idle, *stop;  
    ...  
    u64 clock;  
    ...  
};
```

Overview __schedule

```
static void __sched __schedule(void)
{
    ...
    cpu = smp_processor_id();
    rq = cpu_rq(cpu);
    prev = rq->curr;
    ...
    put_prev_task(rq, prev);
    ...
    next = pick_next_task(rq);
    ...
    if (likely(prev != next)) {
        ...
        rq->curr = next;
        ...
        context_switch(rq, prev, next);
        ...
    }
    ...
}
```

```
static void put_prev_task(struct rq *rq, struct task_struct *prev)
{
    ...
    prev->sched_class->put_prev_task(rq, prev);
}
```

```
#define for_each_class(class) \
    for (class = sched_class_highest; class; class = class->next)

static inline struct task_struct *pick_next_task(struct rq *rq)
{
    const struct sched_class *class;
    struct task_struct *p;
    ...
    for_each_class(class) {
        p = class->pick_next_task(rq);
        if (p)
            return p;
    }
    ...
}
```

... and Finally scheduler_tick()

Just updates the runqueue's clock which checks if the task has occupied the cpu for too long (done by the task_tick in the scheduler class).

scheduler_tick() is periodically invoked by the kernel.

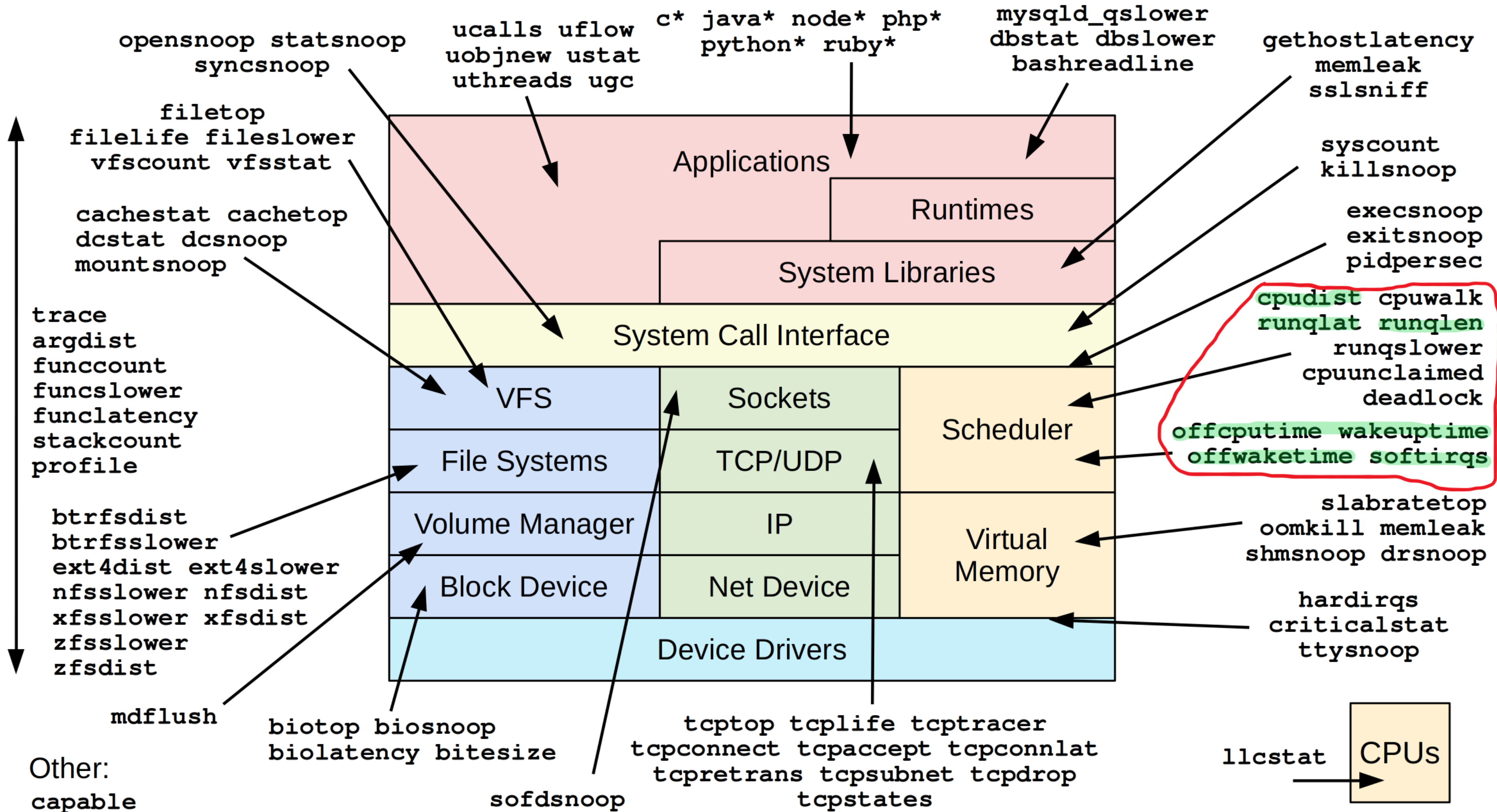
```
void scheduler_tick(void)
{
    int cpu = smp_processor_id();
    struct rq *rq = cpu_rq(cpu);
    struct task_struct *curr = rq->curr;
    ...
    update_rq_clock(rq);
    curr->sched_class->task_tick(rq, curr, 0);
    ...
}
```

CHAPTER III – BCC and BPFTRACE

BCC and BPFTRACE

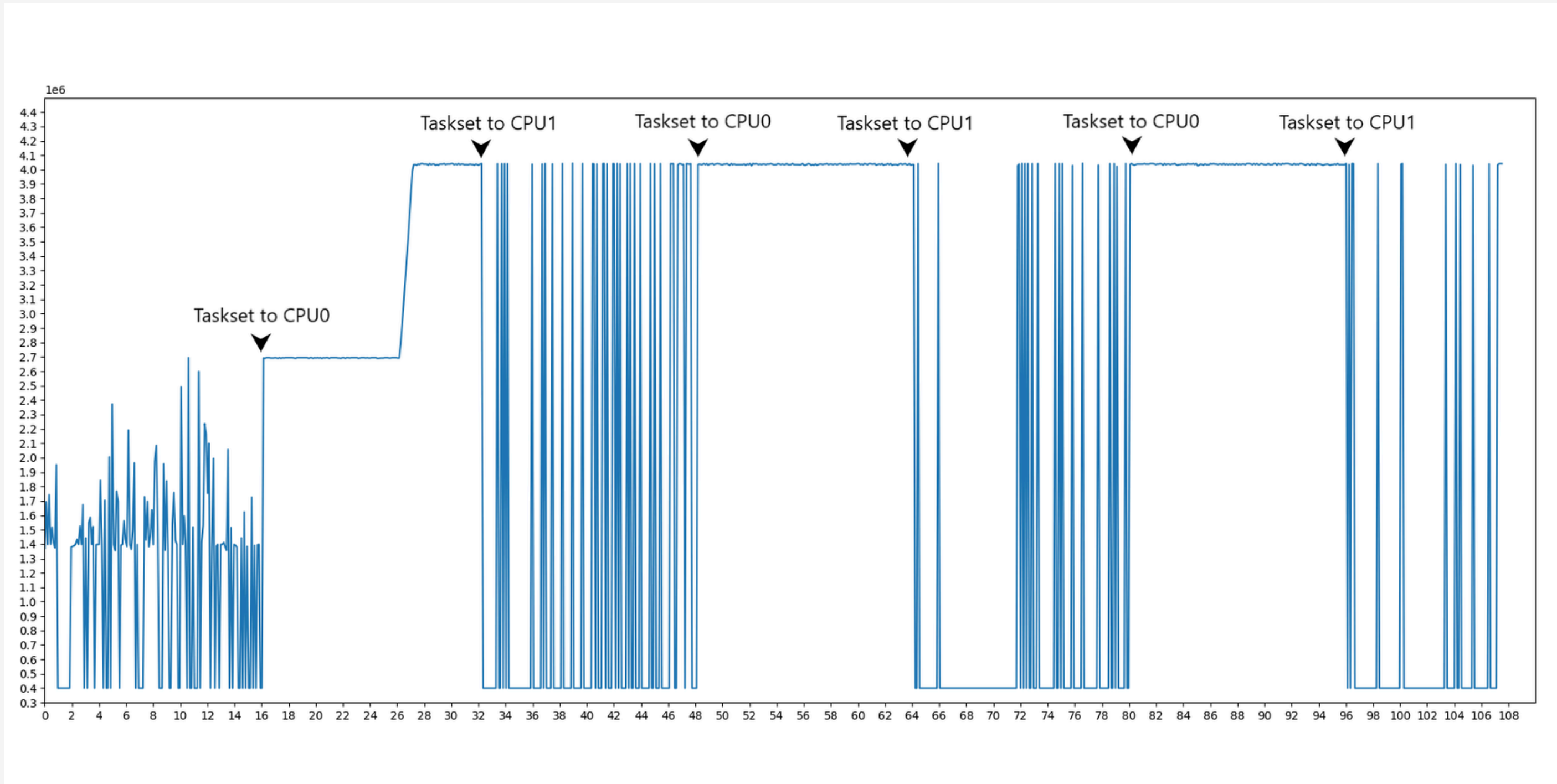
BCC is a toolkit for creating efficient kernel tracing and includes several useful tools and examples. BCC makes BPF programs easier to write, with kernel instrumentation in C.

bpftrace is a high-level tracing language for Linux enhanced Berkeley Packet Filter (eBPF). bpftrace compiles scripts to BPF-bytecode and makes use of BCC for interacting with the Linux BPF system, as well as existing Linux tracing capabilities: kernel dynamic tracing (kprobes), user-level dynamic tracing (uprobes), and tracepoints.



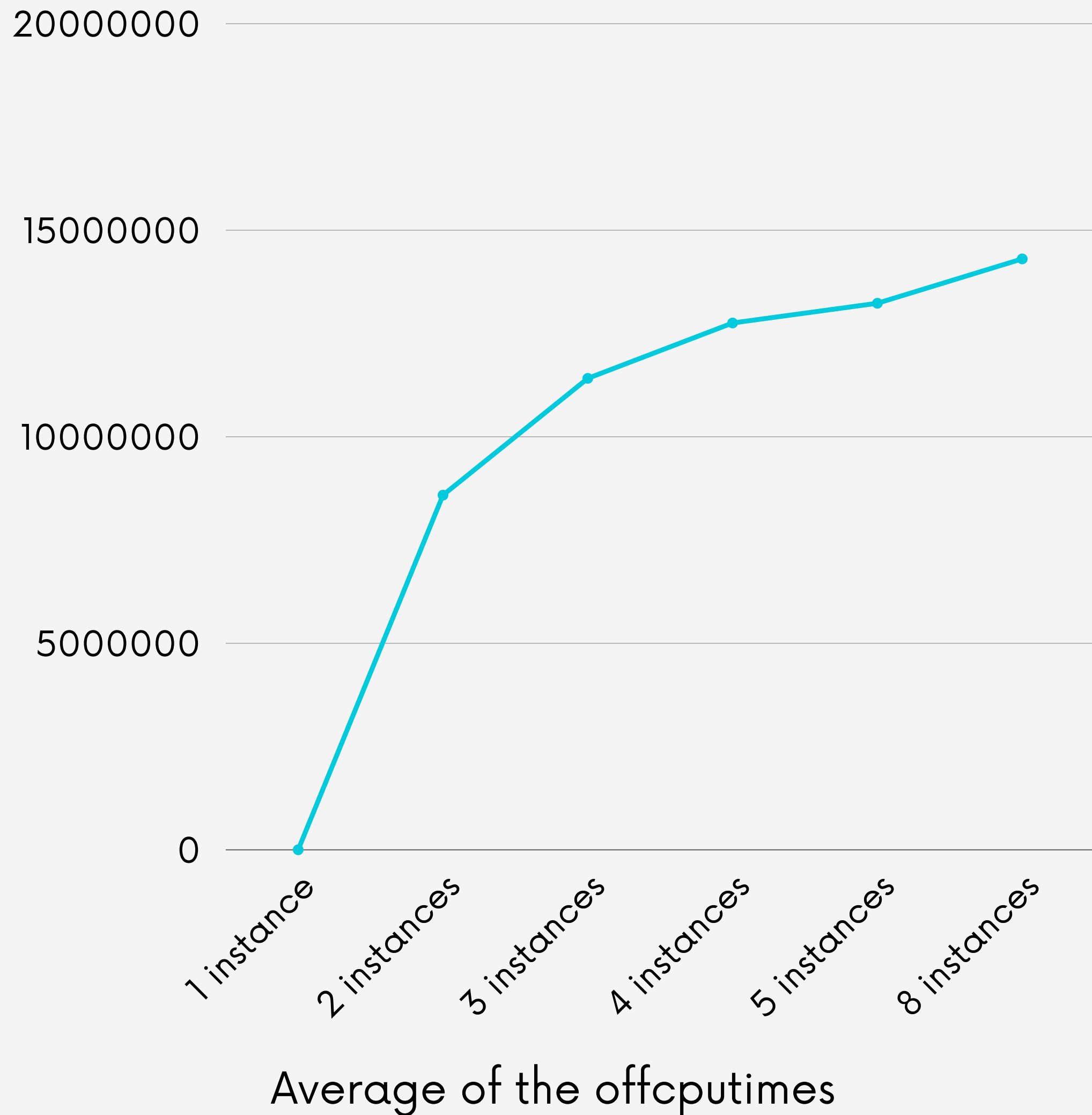
Experiment I

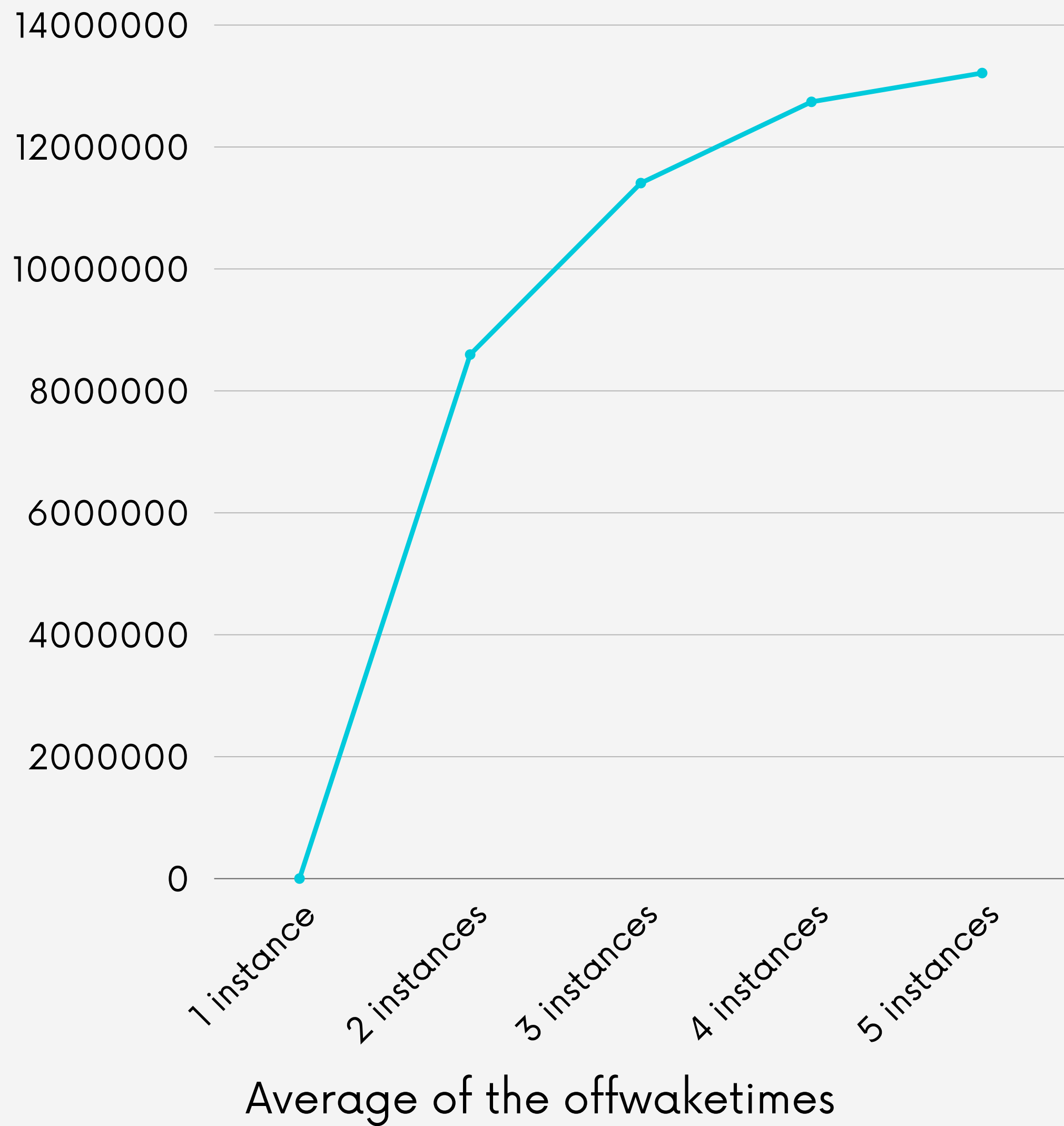
Aim: To visualize the effects of a CPU bound operation on the frequency and utilization of the core by pinning a task to a single core.



Experiment II

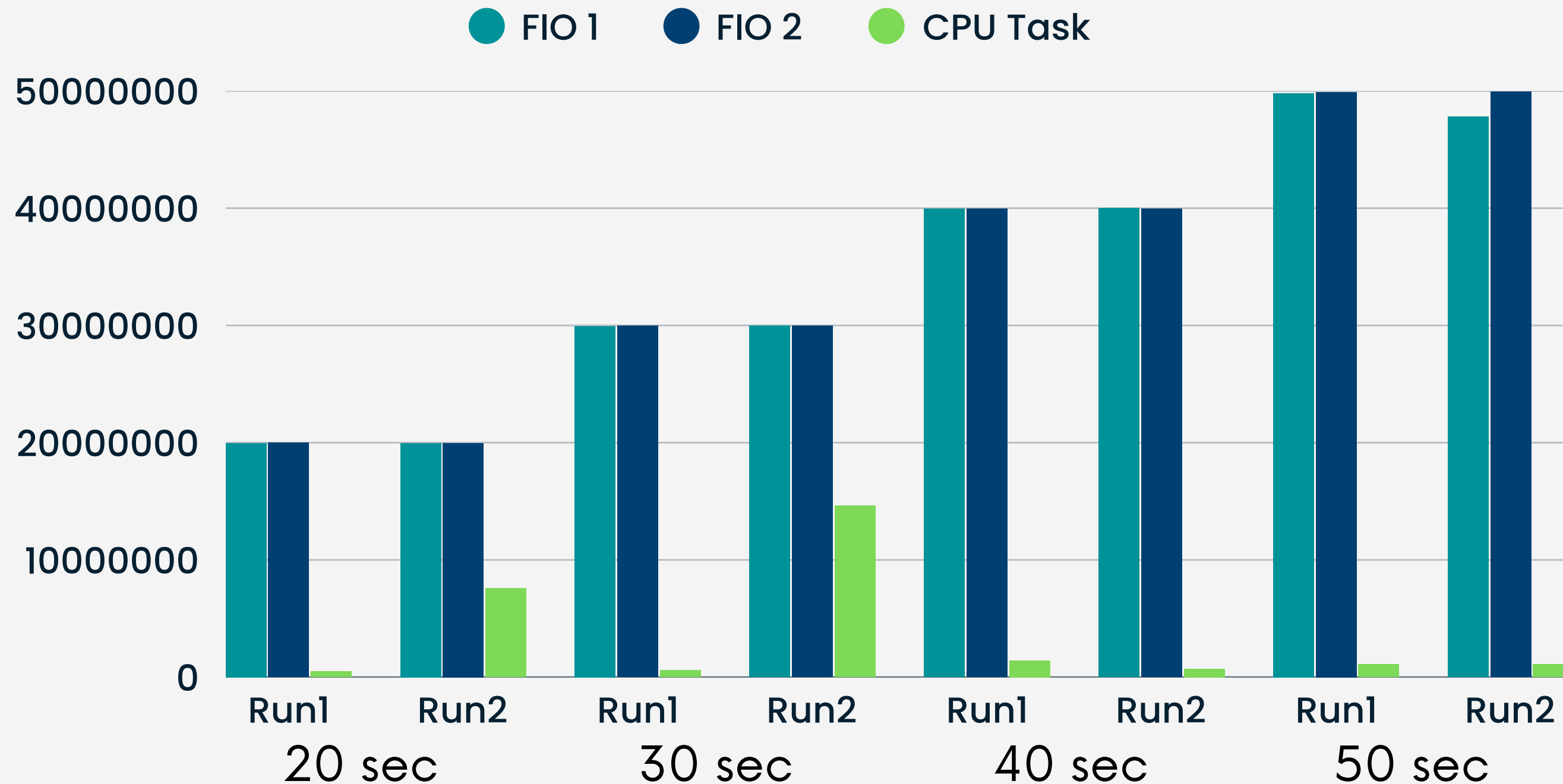
Aim: To generate a CPU bound workloads and pin them to single core and see the effects on the scheduling of the processes.





Experiment III

Aim: To generate a CPU bound workloads, IO workloads with FIO (Flexible IO tester) and look at the effects on scheduling.

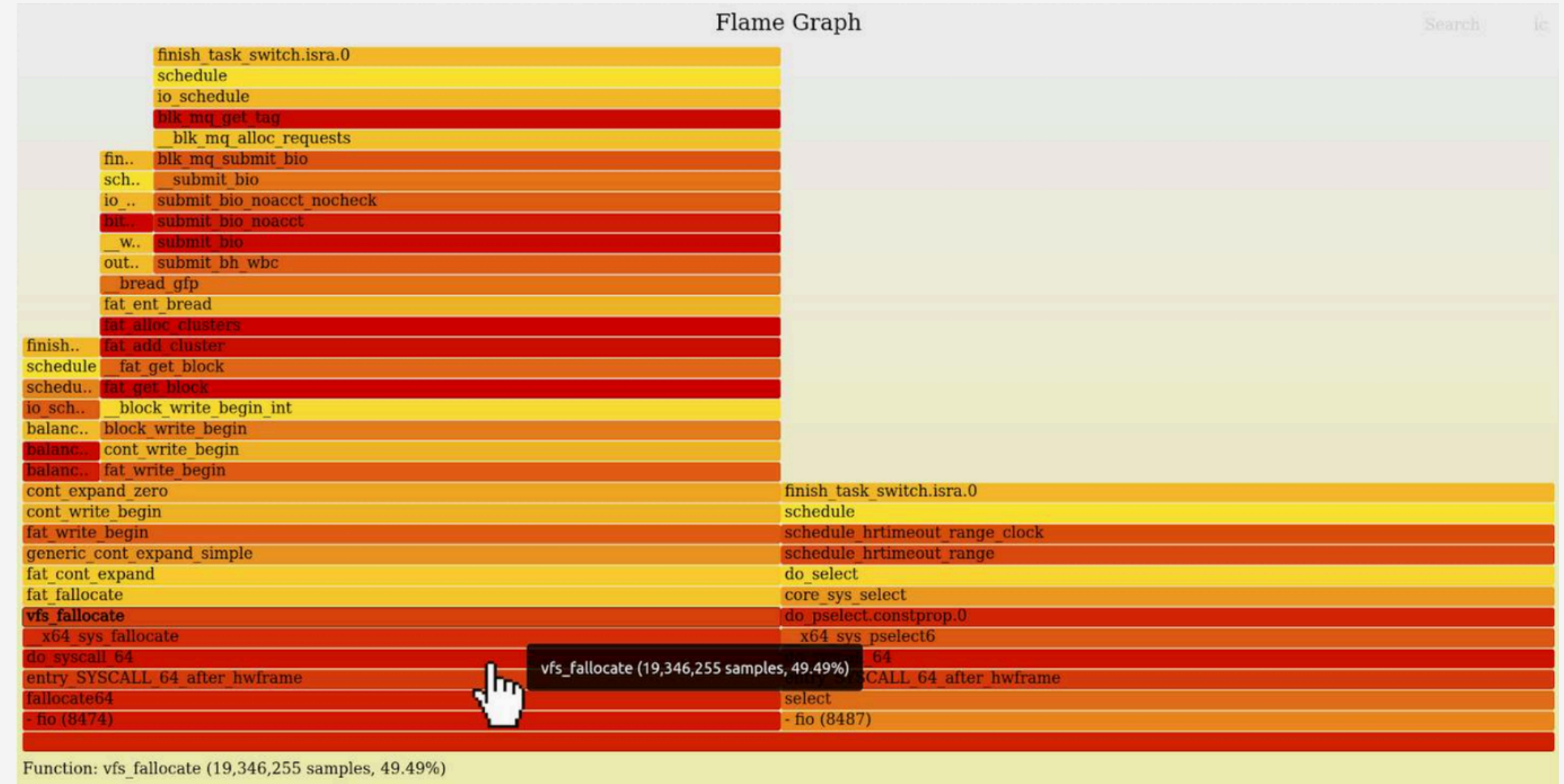


Also generated the stack traces for offcputime, polled runqueue latency and time spent on softirqs.

Visualizing Stacks

Flame graphs are a visualization of hierarchical data, created to visualize stack traces of profiled software so that the most frequent code-paths to be identified quickly and accurately.

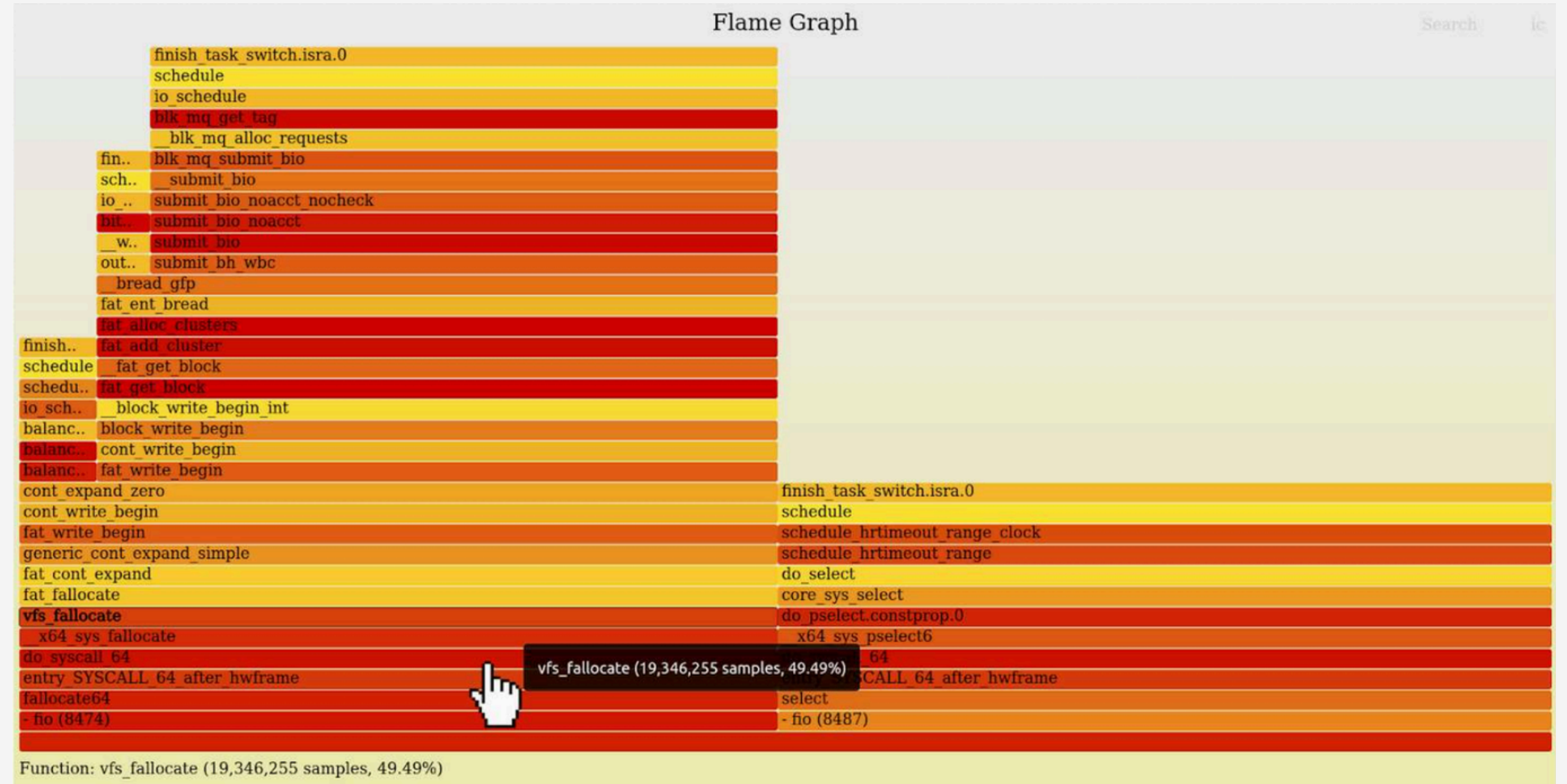
This tool was developed by Brendan Gregg from Netflix to analyze the function call stacks and optimize performance.



Visualizing Stacks

To generate a flame graph, we need to flatten the generated stack trace and process the same via the flamegraph program which then generates an interactive svg graphic.

In our case, the flamegraph of the stack trace shows that `fallocate64`, which is used to allocate memory on the fat file system is called far too many times and such operations, if not optimized, can be a potential bottleneck due to the high latency memory operations have inherently.



THE ROAD AHEAD.....

Having understood the basics of scheduling processes, along with the knowledge of these tracing tools, I must learn to

- Extend the analysis to optimize the applications for better performance.
- Conduct more fine-grained analysis of off-CPU time by focusing on specific subsystems or components of the system. This could involve using more targeted tracing tools and techniques to gain deeper insights into latency issues.
- Look for potential bottlenecks in existing scheduling algorithms and improve on them.

THANK YOU
